ORGNZ-02: Understanding Grail Buckets

> **Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 2 of 10 | **Created:** January 2026 | **Last Updated:** 07/20/2026

Overview

Buckets are logical storage containers in Dynatrace Grail where records are stored. Think of a bucket as a folder in a filesystem. Use buckets to group telemetry data that naturally belongs together, such as data from the same region, environment, or with the same sensitivity classification.

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS environment with Grail enabled
Permissions	`storage:buckets:read` , `storage:logs:read`
Knowledge	Completed ORGNZ-01 (Introduction)
Data	At least 1 hour of log data across default buckets

Table of Contents

Bucket Administration Permissions

Permission	Purpose
`storage:bucket-definitions:read`	View bucket configurations
`storage:bucket-definitions:write`	Create and modify buckets
`storage:bucket-definitions:delete`	Remove buckets
`storage:bucket-definitions:truncate`	Truncate bucket data

Learning Objectives

By the end of this notebook, you will:
- Understand bucket fundamentals and supported data types
- Know bucket limits and query constraints
- Query bucket information using DQL
- Recognize when custom buckets are needed

What Are Buckets?

Buckets organize data to meet performance, compliance, and retention requirements:

Function	Description
Data Grouping	Organize large datasets into meaningful segments

Function	Description
Streamlined Organization	Records that should be handled together are stored together
Query Performance	Reduced queried dataset improves query speed
Retention Management	Each bucket has its own retention policy
Access Control	Bucket-level IAM policies for team isolation
Cost Attribution	Enable data cost tracking by organizational unit

Built-in Grail Buckets

Dynatrace provides a set of predefined built-in buckets that cannot be modified. Use this DQL to discover them all:

```
fetch dt.system.buckets
| filter startsWith(name, "default_") or startsWith(name, "dt_")
```

Bucket Name	Table	Default Retention
`default_logs`	logs	35 days
`default_metrics`	metrics	15 months
`default_spans`	spans	10 days
`default_events`	events	35 days
`default_bizevents`	bizevents	35 days
`default_securityevents`	security.events	1 year
`default_securityevents_builtin`	security.events	3 years
`default_user_events`	user.events	35 days
`default_user_sessions`	user.sessions	35 days
`default_mobile_user_replays`	mobile.user.replays	35 days
`default_web_user_replays`	web.user.replays	35 days

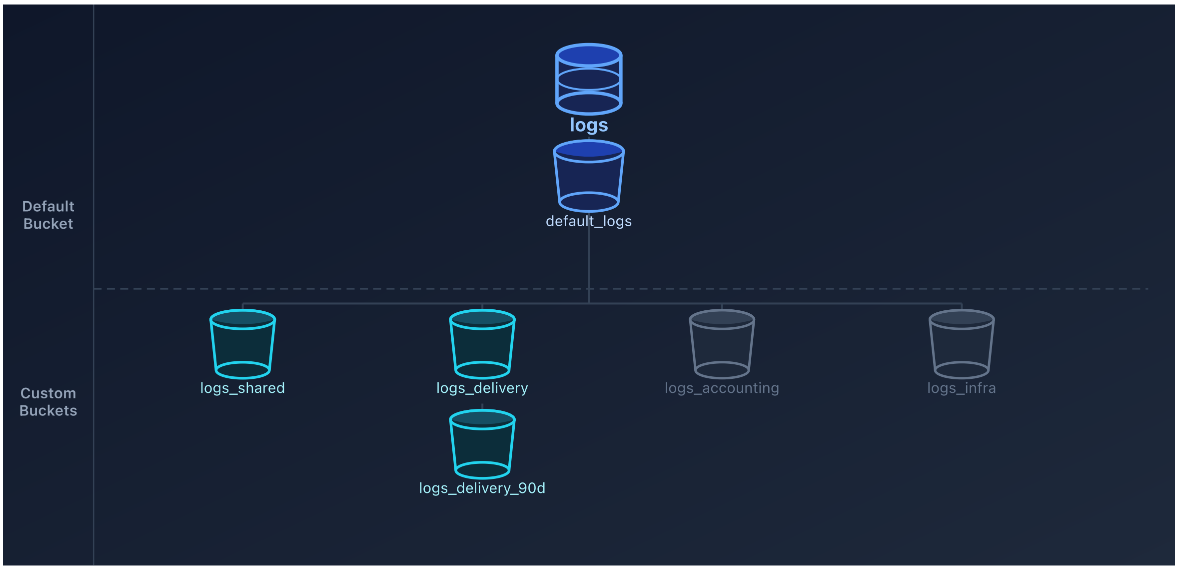
Bucket Name	Table	Default Retention
`default_synthetic_events`	synthetic.events	35 days
`default_synthetic_user_events`	synthetic.user.events	35 days
`default_synthetic_user_sessions`	synthetic.user.sessions	35 days
`default_synthetic_detailed_events`	synthetic.detailed.events	35 days
`default_application_snapshots`	application.snapshots	10 days
`dt_system_events`	dt.system.events	1 year

> **Note:** The `default_securityevents` table was recently migrated to a new Grail security events schema. If you use security events, follow the Grail security table migration guide to complete any required actions.
>
> **Extended retention:** You can extend bucket retention beyond the defaults by joining the Dynatrace preview program. See Preview releases for details.

Supported Data Types (Tables)

Each bucket is associated with exactly **one data type**. You cannot mix data types in the same bucket.

Custom buckets can be created for: logs, events, security events, bizevents, and spans.



> **Important:** Metrics do not support custom buckets — all metric data routes to `default_metrics`.

System Tables

In addition to regular data tables, Grail exposes **system tables** — virtual tables representing Grail metadata, not stored in any bucket:

System Table	Contents
`dt.system.buckets`	Bucket inventory (name, retention, size, table type)
`dt.system.data_objects`	Schema and data object metadata
`dt.system.files`	Lookup files uploaded for DQL enrichment
`dt.system.events`	Audit events (bucket changes, platform actions)

System tables are queried with `fetch dt.system.*` — they never need a `bucket:` parameter.

Bucket Limits

Environment Limits

Limit	Value	Notes
Maximum buckets per environment	80	Default; increase on request (+1 per 10 GB daily ingest)
Typical capacity	Up to 5 TB/day per table	With default bucket limit

Bucket Size Guidelines

Metric	Value	Impact
Optimal ingest per bucket	~1 TB/day	Best query performance
Acceptable ingest	1-2 TB/day	Reduced query window
Split threshold	>2 TB/day	Dynatrace recommends splitting above this

Retention Limits

Limit	Value
Minimum retention	**1 day** for logs, events, and bizevents; **10 days for spans**
Maximum retention	3,657 days (~10 years + 1 week)

Query Constraints

Understanding query limits is critical for bucket planning:

Query Limits

Limit	Value	Impact
Default `fetch` scan limit	500 GB	Default value of the `scanLimitGBytes` parameter — **not** a platform cap. Override per query; `scanLimitGBytes:-1` scans the whole time range
Maximum records returned	1,000	Use aggregations for larger datasets
Maximum response payload	1 MB	Large result sets may be truncated

Queryable Window by Ingest Volume

With the default 500 GB `fetch` scan limit, a single unmodified query reaches back roughly:

Daily Ingest	Queryable Window
500 GB/day	~24 hours
1 TB/day	~12 hours
3 TB/day	~4 hours
10 TB/day	~1 hour

> **Implication**: High-volume buckets may only allow querying recent data. Use aggregations, time filters, and strategic bucket partitioning to work within these limits.

Default Buckets

Dynatrace provides default buckets with varying retention:

Default Bucket	Data Type	Default Retention	Notes
`default_logs`	logs	35 days	Standard log retention
`default_metrics`	metrics	15 months	Extended for trend analysis
`default_spans`	spans	10 days	Short-term APM data
`default_events`	events	35 days	Platform events
`default_bizevents`	bizevents	35 days	Business events
`default_securityevents`	security_events	1 year	Security events
`default_securityevents_builtin`	security_events	3 years	Built-in security events

> **Note**: Query your environment's `dt.system.buckets` to verify current retention settings.

Querying Bucket Information

Querying Data from Buckets

Bucket Characteristics

Immutability Rules

Characteristic	Mutable?	Notes
Bucket name	No	Cannot be changed after creation
Display name	Yes	Can be updated anytime
Retention period	Yes	**Caution**: Lowering deletes data
Data table type	No	Fixed at creation
Bucket class (live / historic)	No	Read-only after creation

Data Handling

Operation	Supported?	Alternative
Move data between buckets	No	Route new data via OpenPipeline
Selective record deletion	No	Wait for retention to expire
Bucket deletion	Yes	Deletes ALL data permanently

Bucket Naming Rules

Rule	Requirement
Name length	3–100 characters
First character	Must be a letter
Allowed characters	Lowercase alphanumeric, underscores, hyphens
Case	Lowercase only
Reserved names	Cannot use `default_*` prefix

Valid Examples

```
team_logs_30d
finance-audit-logs
prod_metrics_90d
app123_spans
```

Invalid Examples

```
123_logs          # Starts with number
Team_Logs         # Contains uppercase
default_custom    # Uses reserved prefix
my logs           # Contains space
```

When to Create Custom Buckets

Need	Solution
Different retention periods	Create bucket with specific retention
Cost attribution by team/LOB	Separate buckets per cost center
Simple team isolation	Bucket-level IAM policies
Regulatory compliance	Dedicated buckets for compliance data
Query performance	Isolate high-volume data sources
Debug/verbose logs	Short-retention bucket to reduce costs

Next Steps

Continue with the ORGNZ series:
- **ORGNZ-03**: Bucket Strategy and Design

References

- Grail Buckets
- Data Retention
- DQL Reference

This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.

DQL: Built-in Bucket Discovery

Discover all built-in Grail buckets in your environment:

```
// List all built-in Grail buckets — inventory check
fetch dt.system.buckets
| filter startsWith(name, "default_") or startsWith(name, "dt_")
| fields name, display_name, dt.system.table, retention_days
| sort dt.system.table asc, name asc
```

DQL: Bucket Audit Events

Track who created, updated, truncated, or deleted a bucket:

```
// Audit bucket management actions — returns create, update, truncate, delete
events
fetch dt.system.events
| filter event.kind == "AUDIT_EVENT" and event.category ==
"BUCKET_MANAGEMENT"
| sort timestamp desc
```

DQL: Querying Data from Buckets

Use the `bucket:` parameter to target specific storage:

```
// Query logs from a specific bucket — use the bucket: parameter to target
storage
fetch logs, from:-1h, bucket:"default_logs"
| limit 10
```

```
// Query from multiple buckets simultaneously — controls scan scope
fetch logs, from:-1h, bucket:{"default_logs", "audit_logs", "security_logs"}
| limit 100
```